Static Quantization Is Not a Cross-Platform Reproducibility Primitive: A Measurement Study and an Integer-Only Design

James Burrell JJB2271@COLUMBIA.EDU
Columbia University

Editors: Andy Song, Bo Han and Sarah Erfani

Abstract

Floating-point Transformer inference is usually stable enough to deploy, but it promises no bitwise equality across batch shapes, kernels, or hardware platforms. It is tempting to treat integer quantization as a fix: an INT8×INT8→INT32 matmul accumulates in exact integer arithmetic, independent of reduction order. We test this claim with a frozen 286M-parameter NanoChat checkpoint and a preregistered matrix of seven arithmetic methods across batch sizes 1, 8, and 64, with 1,000 greedy generations per condition, replicated on a second (CPU) hardware platform. We report three results. First, a *null*: on a single GPU, batch shape induces no output nondeterminism for any precision: greedy FP32 already matches its own batch-1 reference on 100% of prompts. Second, a *nuance*: static W8A8 makes the logits *exactly* batch-invariant (maximum logit error identical to eight significant figures across batch 1/8/64), and this holds for both static and dynamic scaling, so the mechanism is integer accumulation, not scale freezing. Third, and centrally, a *negative result*: this invariance does not extend across platforms and in fact reverses. Cross-platform generation agreement (GPU vs. CPU) falls from 100% for FP32 to 85% (BF16), 45% (dynamic W8A8), and 25% (static W8A8), because normalization, softmax, rotary embeddings, and dequantization remain floating point and INT8 rounding packs logits near decision boundaries. We conclude that quantizing only the linear layers cannot be a cross-platform reproducibility primitive, and we specify the integer-only design (an op-by-op integerization trained with quantization-aware training) that would convert batch-invariance into platform-invariance.

Keywords: quantization; reproducibility; determinism; integer-only inference; Transformers

1. Introduction

Practitioners routinely assume that a fixed model, checkpoint, and prompt yield a fixed output. In floating point this is false in a subtle way: the IEEE-754 operations used by matrix multiplication are not associative, so the order in which partial sums are accumulated (which depends on batch shape, the kernel a library selects, and the hardware it runs on) can change the result in the last few bits (PyTorch Team, a). Those bits occasionally propagate through an arg max and change a generated token. Fixing a random seed does not help: seeding controls sampling, not the numerical path of the forward pass (PyTorch Team, b; Thinking Machines Lab, 2025).

Integer quantization is usually motivated by memory and throughput. It also has an appealing reproducibility property: an integer matmul accumulates in exact `int32` arithmetic

and is therefore *independent* of accumulation order. This paper asks whether that property makes static weight-and-activation 8-bit quantization (W8A8) a *reproducibility primitive*: does a fixed quantization grid reduce a Transformer's output sensitivity to batch shape and to a change of hardware platform, relative to FP32/FP16/BF16, and if so, why?

This is a measurement paper with a working prototype, not a completed integer-only Transformer. Our prototype quantizes only the aligned core linear layers to INT8×INT8→INT32 and dequantizes for every nonlinear and residual operation; we use the label **static W8A8 linear** throughout to keep that boundary explicit. Our contributions are:

- A preregistered measurement of quantization as a reproducibility mechanism, with operational definitions separating determinism, reproducibility, and fidelity.
- Evidence that integer matmul yields *exact* batch-invariance of logits, and that the mechanism is integer accumulation rather than scale freezing.
- A cross-platform (GPU vs. CPU) result showing that partial (linear-only) quantization *degrades* rather than improves reproducibility.
- A concrete integer-only design (an op-by-op integerization of a non-standard architecture, trained with quantization-aware training) that would extend batch-invariance to cross-platform determinism.

2. Motivation and Problem Statement

Definitions. We distinguish four properties. *Determinism*: the same process, run twice, produces identical output. *Repeatability*: identical output across fresh process launches on the same machine. *Reproducibility*: identical output across batch shapes, kernels, or platforms. *Fidelity*: closeness of the output distribution to a chosen FP32 reference. A method can be perfectly repeatable yet unfaithful, and this distinction drives our results.

Why batch shape and platform matter. A decode step for a single sequence and the same step inside a batch of 64 traverse different kernels and reduction trees; a step on a GPU and on a CPU traverse entirely different implementations. Because floating-point addition is non-associative and transcendental libraries differ, the resulting logits can differ. Whether that difference changes a token is an empirical question, which motivates our design: we hold the model, checkpoint, prompt, and decoding schedule fixed and vary *only* the physical batch shape, the arithmetic method, and the hardware platform.

Research question. Does calibration-fixed W8A8 linear inference reduce NanoChat's sensitivity to batch shape and to platform relative to floating point, and is any reduction attributable to fixed scales rather than integer matrix multiplication alone? We do not equate fixed seeds with deterministic inference, and we do not claim cross-platform determinism without measuring it.

3. Related Work

Integer-only Transformers. I-BERT (Kim et al., 2021) approximates GELU, softmax, and layer normalization with integer and dyadic arithmetic so that BERT inference is

integer-only; I-ViT (Li and Gu, 2023) extends this to vision Transformers with integer attention and normalization. BitNet (Wang et al., 2023) is the most integer-centric training recipe at LLM scale, learning 1-bit weights from scratch, but its goal is efficiency: activations, normalization, and attention remain in higher-precision floating point, so cross-platform bitwise determinism is not among its claims. These works target accuracy and latency, and they establish that the nonlinear operations we leave in floating point *can* be made integer, which is precisely the design we specify in Section 7. The integer-arithmetic training of Jacob et al. (2018) underpins modern integer inference.

LLM post-training quantization. LLM.int8() (Dettmers et al., 2022) and SmoothQuant (Xiao et al., 2023) make W8A8 accurate at scale by handling activation outliers with per-token/per-channel scales and outlier migration; ZeroQuant (Yao et al., 2022) adds group-wise scaling with layer-by-layer distillation, OmniQuant (Shao et al., 2024) learns clipping and equivalent transformations for weights and activations, and QuaRot (Ashkboos et al., 2024) rotates activations to eliminate outliers, enabling full 4-bit quantization of weights, activations, and the KV cache. A complementary line compresses *weights only*: GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024), SpQR (Dettmers et al., 2024), and AQLM (Egiazarian et al., 2024) reach 2–4 bit weights but keep activations, and hence the matmul accumulation itself, in floating point; they therefore cannot inherit the order-invariance of integer accumulation, and we scope them out of this study for that reason. Serving systems such as Atom (Zhao et al., 2024) and QServe (Lin et al., 2025) do execute low-bit integer kernels end-to-end, but they select kernels by batch size and hardware to maximize throughput, which is exactly the variability we measure here. This literature optimizes accuracy, memory, and throughput; to our knowledge it does not measure *generation reproducibility* as the primary outcome. Our prototype uses deliberately minimal per-tensor static activation quantization to isolate the reproducibility question, which is why its fidelity is below these accuracy-oriented methods; Section 7 adopts their per-token scheme.

Cross-platform consistency and framework guidance. Cross-platform post-training quantization has been studied where encoder/decoder mismatch is catastrophic, e.g. learned image compression (Yu et al., 2022). Vendor documentation warns that batched and sliced computations need not be bitwise equal and that determinism requires explicit, performance-costly flags (PyTorch Team, a,b). Recent systems work makes LLM inference batch-invariant by controlling reductions (Thinking Machines Lab, 2025); we instead ask whether quantization achieves invariance as a side effect of integer accumulation, and find that partial quantization does not achieve it across platforms.

4. NanoChat W8A8 Prototype

Model. We use a frozen NanoChat (Karpathy, 2025) base checkpoint (`paper-d12`, step 2520; 12 layers, d=768, 6 heads, vocabulary 32768; 286,261,730 parameters; trained on 1.321B tokens; validation BPB 0.8396). The checkpoint, tokenizer, and prompt/calibration manifests are pinned by SHA-256 and archived with every result directory.

Arithmetic conditions. FP32 is literal float32 with TF32 disabled; FP16 and BF16 use NanoChat's explicit half-precision paths; deterministic FP32/FP16 controls enable PyTorch deterministic algorithms with a fixed cuBLAS workspace. The static W8A8 prototype

Algorithm 1: Static W8A8 linear (per-channel weights, per-tensor frozen activation).
Input: activation x; frozen scales s_a (per-tensor), s_w (per-channel); int8 weights W_q; bias b
Calibrate (offline): run the float model on 32 strings; set $s_a = \max|x|/127$ $x_q \leftarrow \text{clip}(\text{round}(x/s_a), -127, 127)$; `// quantize to int8`
if rows$(x_q) < 32$ **then** zero-pad x_q to 32 rows (`_int_mm` constraint);
$A \leftarrow$ `_int_mm`$(x_q, W_q^\top)$; `// int8 × int8 → int32`
return $A_{[:\text{rows}]} \cdot (s_a s_w) + b$; `// dequantize in fp32`

(Algorithm 1) uses symmetric INT8 with per-output-channel weight scales and a per-tensor activation scale frozen from 32 calibration strings; it executes INT8×INT8→INT32 with PyTorch's `_int_mm` and dequantizes the `int32` accumulator. The dynamic W8A8 control uses the identical integer matmul but re-estimates the activation scale at runtime; comparing the two isolates whether any stability comes from the fixed grid or from integer arithmetic alone.

Claim boundary. Only aligned core projections are quantized (in/out features divisible by 8). NanoChat's tiny unaligned smear and value-gate projections, and all normalization, rotary embedding, attention softmax, residual, and dequantization operations, remain floating point. This isolates grid snapping and integer accumulation but is *not* integer-only inference. For decode matmuls with 16 or fewer rows (which PyTorch's CUDA `_int_mm` rejects), both W8A8 paths zero-pad to 32 rows and slice the padding from the output.

5. Experimental Protocol

The protocol is preregistered and frozen. For every arithmetic method × batch size $\in \{1, 8, 64\}$ we retain exactly 1,000 greedy generations: 20 prompts × 10 replicas × 5 fresh process launches. The final partial logical batch is padded to the declared physical batch size and padding rows are discarded. The FP32 batch-1 teacher-forced trace is the common reference. We then replicate the four core methods (FP32, BF16, static/dynamic W8A8) on a second hardware platform (CPU), where the integer matmul executes but every surrounding kernel differs from the GPU.

Outcomes. Generation: unique sequences per prompt across restarts, FP32 batch-1 reference-match rate, modal fraction, and, for cross-platform, whether the modal token sequence is identical across GPU and CPU. Numerical: mean and maximum absolute logit difference, logit RMSE, and forward KL from the FP32 batch-1 distribution. Fidelity: held-out validation bits-per-byte (BPB) over 2,097,152 tokens, throughput, and a static calibration-size sensitivity at 1/8/32 samples.

Statistics. We report prompt-level means and 95% prompt-cluster bootstrap intervals (10,000 resamples). Replicas and process launches are repeated measurements, not independent prompts. Every prespecified condition is reported, including null results.

Hypotheses. **H1** (sanity): FP16 deviates from the FP32 batch-1 reference more than FP32 at batch 8/64. **H2** (central): static W8A8 has lower within-condition output multiplicity than FP16 under batch perturbation. **H3**: static W8A8 may raise reference-match

Table 1: Greedy matrix (1,000 generations per row). $|\Delta|$ is absolute logit difference from the FP32 batch-1 reference; KL is forward KL in nats; BPB is held-out validation fidelity. Static/dynamic W8A8 use calibration 32.

Method	Batch	Match	Mean$\|\Delta\|$	Max$\|\Delta\|$	KL	BPB
FP32	1	1.00	0.00e+00	0.00e+00	2.18e-09	0.8395
FP32	8	1.00	1.08e-06	1.32e-05	2.29e-09	0.8395
FP32	64	1.00	1.74e-06	1.62e-05	5.23e-09	0.8395
FP16	1	0.95	3.22e-03	2.34e-02	2.31e-06	0.8395
FP16	8	0.95	3.21e-03	2.33e-02	2.33e-06	0.8395
FP16	64	0.95	3.19e-03	2.42e-02	2.33e-06	0.8395
BF16	1	0.75	2.56e-02	1.83e-01	1.46e-04	0.8396
BF16	8	0.70	2.56e-02	1.80e-01	1.46e-04	0.8396
BF16	64	0.70	2.57e-02	1.85e-01	1.47e-04	0.8396
Static W8A8	1	0.00	6.92e-01	4.88e+00	9.55e-02	0.8752
Static W8A8	8	0.00	6.92e-01	4.88e+00	9.55e-02	0.8752
Static W8A8	64	0.00	6.92e-01	4.88e+00	9.55e-02	0.8752
Dynamic W8A8	1	0.15	3.52e-01	3.48e+00	3.42e-02	0.8761
Dynamic W8A8	8	0.15	3.52e-01	3.48e+00	3.40e-02	0.8761
Dynamic W8A8	64	0.15	3.52e-01	3.48e+00	3.40e-02	0.8761

rate while increasing KL, separating reproducibility from fidelity. **H4**: static is more stable than dynamic W8A8 if scale freezing, rather than integer matmul, is the mechanism.

6. Results

A same-machine null result (H1, H2). On one GPU, batch shape induces *no* output nondeterminism. Every method at every batch size has unique output count 1.0 (Table 1): each prompt yields a single sequence across all replicas and launches. FP32 matches its own batch-1 reference on 100% of prompts at batch 8/64 despite a nonzero maximum logit drift of $\sim 1.6 \times 10^{-5}$; that drift never crosses an arg max boundary. H1 holds directionally, but H2 finds no multiplicity to reduce: FP16 and BF16 are already output-deterministic across batch on this machine.

Exact batch-invariance from integer matmul (H4). Where floats carry small batch-dependent noise, the integer paths do not. Static W8A8's maximum logit error is identical to eight significant figures across batch 1/8/64 (Table 1, Figure 1), so its logits are exactly invariant to batch shape, which FP32 is not. Dynamic W8A8 is likewise invariant. Because both the fixed-scale and runtime-scale variants are exactly invariant, **H4 is not supported**: the reproducibility mechanism is exact integer accumulation, not scale freezing.

Reproducibility is not fidelity (H3). The invariance is purchased by computing a different function. Static W8A8 matches the FP32 reference on 0% of prompts and dynamic on 15%; static W8A8's mean KL from FP32 is 0.095 nats versus 2.3×10^{-6} for FP16. Validation BPB rises from 0.8395 (FP32) to 0.8752 (static) and 0.8761 (dynamic), whereas

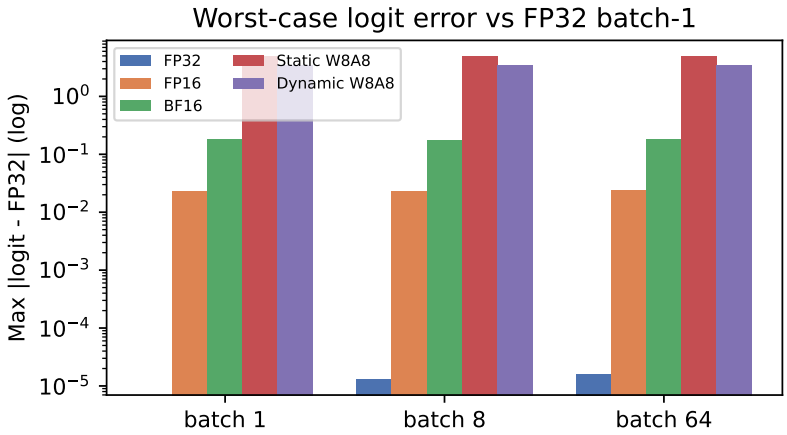

Figure 1: Worst-case logit error vs. FP32 batch-1 (log scale). FP32 batch-1 is zero by construction; float error grows slightly with batch, while W8A8 error is large but *constant* across batch shape.

Table 2: Fidelity ablation: held-out validation BPB over 2.1M tokens.

Method	Calib. samples	Validation BPB
BF16	–	0.83959
Dynamic W8A8	–	0.87612
FP16	–	0.83954
FP32	–	0.83954
Static W8A8	1	0.87715
Static W8A8	8	0.87509
Static W8A8	32	0.87520

FP16/BF16 stay within 10^{-4} of FP32 (Table 2). H3 is confirmed: a W8A8 model can be perfectly repeatable and batch-invariant while being a worse approximation of FP32.

Where precision flips tokens. Reproducibility risk is content-dependent (Table 3). Under FP16/BF16, factual, numerical, reasoning, and instruction prompts match the FP32 reference on 100% of cases, while *completion* and *ambiguous* prompts flip most often (BF16 33% each). These are prompts whose top logits are nearly tied, so a 10^{-2}-scale perturbation crosses the decision boundary. With two or three prompts per category this is a descriptive signal, but it indicates that any reproducibility budget should be spent on open-ended generation rather than factual recall.

The central negative result: cross-platform (CPU). We replicate the core methods on a CPU platform and measure whether the modal generated token sequence per prompt is identical on GPU and CPU (Table 4, Figure 2). The outcome is the opposite of the naive expectation. FP32 reproduces the same text on **100%** of prompts across platforms: at full precision the GPU↔CPU logit gap never crosses an arg max boundary. Agreement then *falls* as precision drops: BF16 85%, dynamic W8A8 45%, static W8A8 only **25%**.

Table 3: Per-category reference-match rate (batch 1). Lower means the precision flips more tokens relative to FP32.

Category	FP16	BF16	Static W8A8	Dynamic W8A8
Factual	1.00	1.00	0.00	0.00
Reasoning	1.00	1.00	0.00	0.33
Numerical	1.00	1.00	0.00	0.50
Technical	1.00	0.67	0.00	0.00
Instruction	1.00	1.00	0.00	0.00
Completion	0.67	0.33	0.00	0.00
Ambiguous	1.00	0.33	0.00	0.33

Table 4: Cross-platform replication: fraction of prompts whose modal greedy token sequence is identical on GPU and CPU (batch 1).

Method	GPU↔CPU text match
FP32	100.0%
BF16	85.0%
Static W8A8	25.0%
Dynamic W8A8	45.0%

Partial quantization is thus markedly *worse* for cross-platform reproducibility than full precision. The mechanism compounds two effects: integer accumulation removes batch-dependent variation, but INT8 rounding packs many logits near decision boundaries, and the surrounding floating-point operations (dequantization, normalization, softmax, rotary) use different kernels on CPU than on GPU. No method here produces bit-identical logits across platforms; FP32 merely has enough headroom that its argmax is unchanged.

7. Design: An Integer-Only Path to Platform Invariance

The negative result has a clear cause: only the linear layers are integer, so the argmax-determining path still runs through platform-divergent floating-point operations. We specify the design that would remove them. It is a design and feasibility analysis; we have not yet trained or evaluated it.

Determinism, not compression, sets the bit-widths. Because the goal is identical bits rather than smaller tensors, only the large matmuls need to be INT8; the nonlinear operations can use wide INT32 fixed-point, which preserves accuracy while remaining exactly reproducible. Table 5 maps each NanoChat operation to its integer replacement. Two operations are effectively free: relu^2 is exact in integer, and the monotone `tanh` softcap does not change a greedy argmax and can be dropped.

Quantization-aware training with train/inference parity. Post-training integerization of a checkpoint not trained for it risks incoherent output on a weak 286M model: "deterministic garbage." The accurate route is quantization-aware training (QAT): simulate integer rounding in the forward pass with a straight-through estimator while keeping

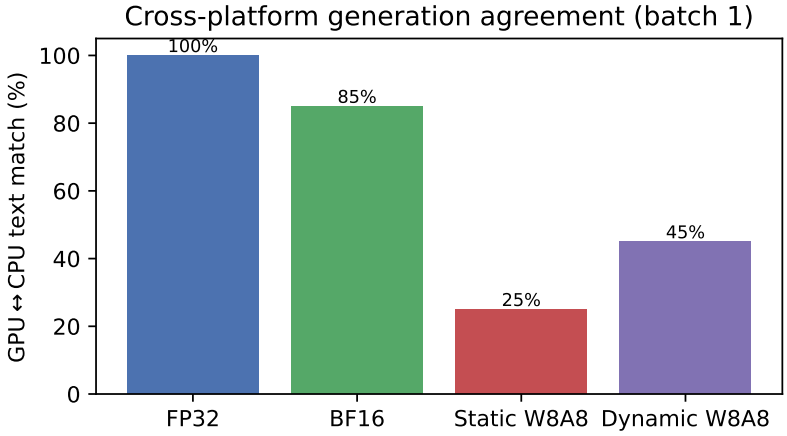

Figure 2: GPU↔CPU generation agreement. Full precision is already platform-stable at the token level; lower precision diverges, and linear-only quantization is the worst. The remaining floating-point operations prevent bitwise platform invariance.

Table 5: Op-by-op integerization of NanoChat. Widths are chosen for determinism.

Operation	Float today	Integer target
Core linears	fp matmul	INT8×INT8→INT32 + dyadic requant
RMSNorm (no-param)	`rms_norm`	integer inverse-sqrt (Newton), int32
Attention $QK^\top/\times V$	fp (flash-attn)	integer matmul + requant
Softmax	fp exp/÷	I-BERT integer exp + integer normalize
Rotary x_1c+x_2s	fp tables	fixed-point cos/sin (Q15)
relu^2	fp	integer square (exact)
sigmoid gates	fp	integer LUT
resid/x0/backout λ	fp mul	dyadic fixed-point scalar
Embeddings	fp table	per-channel int8 table
softcap $15\tanh(z/15)$	fp	dropped (monotone ⇒ argmax-preserving)
Logits	fp	int32 (argmax directly)

fp32 master weights for the optimizer, using per-token activation scales (which, unlike our per-tensor prototype, control outliers). The non-negotiable correctness requirement is *parity*: the fake-quant forward used in training must be arithmetically identical to the integer inference path, so the trained model is exactly the deployed model. A warm-started QAT finetune from `paper-d12` for a fraction of the original tokens is the low-cost first step.

Expected outcome and the determinism metric. If the entire path is integer, the int32 logits are computed with no non-associative float reductions and no platform-specific transcendental libraries, so the SHA-256 hash of the logit tensor should be identical across batch 1/8/64 and across GPU/CPU, while every method in Table 4 differs. This converts the batch-invariance we already observe (Table 1) into the platform-invariance we do not.

8. Discussion, Threats, and Roadmap

A null result is informative. For this model on this GPU, greedy same-machine inference is already batch-invariant at the token level; quantization is not needed to stabilize same-machine batching, and using it for that purpose forfeits fidelity for no reproducibility gain. The defensible positive claim is narrower and exact: integer matmul yields bitwise-identical logits across batch shapes, which matters for applications that compare log-probabilities rather than only tokens.

Threats to validity. (i) One model, one checkpoint, and two platforms cannot establish generality. (ii) FP32's same-machine and cross-platform stability is regime-dependent (small model, greedy decoding, short 32-token generations, high precision); it would not survive sampling, long sequences, or near-tie-heavy prompts, as the category analysis suggests. (iii) Our prototype uses minimal per-tensor activation quantization, so its cross-platform result partly reflects quantization quality; however, the floating-point islands are a barrier that better quantization cannot remove, which is why Section 7 integerizes them. (iv) The padding compatibility path and the remaining float operations are confounds for any strict determinism claim.

Roadmap. Build the integer-only path of Section 7, verify train/inference parity, train a QAT checkpoint to fidelity within $\sim$0.02 BPB of FP32, and re-run this matrix plus a 1,000-completion end-to-end test, reporting logit-hash agreement across batch and platform.

9. Conclusion

Studied as a reproducibility primitive, static W8A8 delivers exactly what integer accumulation promises (logits invariant to batch shape) but no more: it is a different, lower-fidelity function, and its invariance not only fails to survive a change of hardware platform but reverses, dropping cross-platform text agreement from 100% (FP32) to 25%, because most of the network is still floating point and INT8 logits sit near decision boundaries. Batch-invariance and platform-invariance are distinct goals, and only the former follows from integer matmul alone. A genuinely integer-only inference path, trained with quantization-aware training, is the route from one to the other, and is the natural next step.

References

Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.00456.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A

sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.03078.

Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.06118.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. https://github.com/karpathy/nanochat, 2025.

Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. I-BERT: Integer-only BERT quantization. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2101.01321.

Zhikai Li and Qingyi Gu. I-ViT: Integer-only quantization for efficient vision transformer inference. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv:2207.01405.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Machine Learning and Systems (MLSys)*, 2024. arXiv:2306.00978.

Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Machine Learning and Systems (MLSys)*, 2025. arXiv:2405.04532.

PyTorch Team. PyTorch documentation: Numerical accuracy. https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html, a. Accessed 2026.

PyTorch Team. PyTorch documentation: Reproducibility. https://docs.pytorch.org/docs/stable/notes/randomness.html, b. Accessed 2026.

Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.13137.

Thinking Machines Lab. Defeating nondeterminism in LLM inference. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/, 2025.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2206.01861.

Dailan Yu et al. Post-training quantization for cross-platform learned image compression. *arXiv preprint arXiv:2202.07513*, 2022.

Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. Atom: Low-bit quantization for efficient and accurate LLM serving. In *Machine Learning and Systems (MLSys)*, 2024. arXiv:2310.19102.